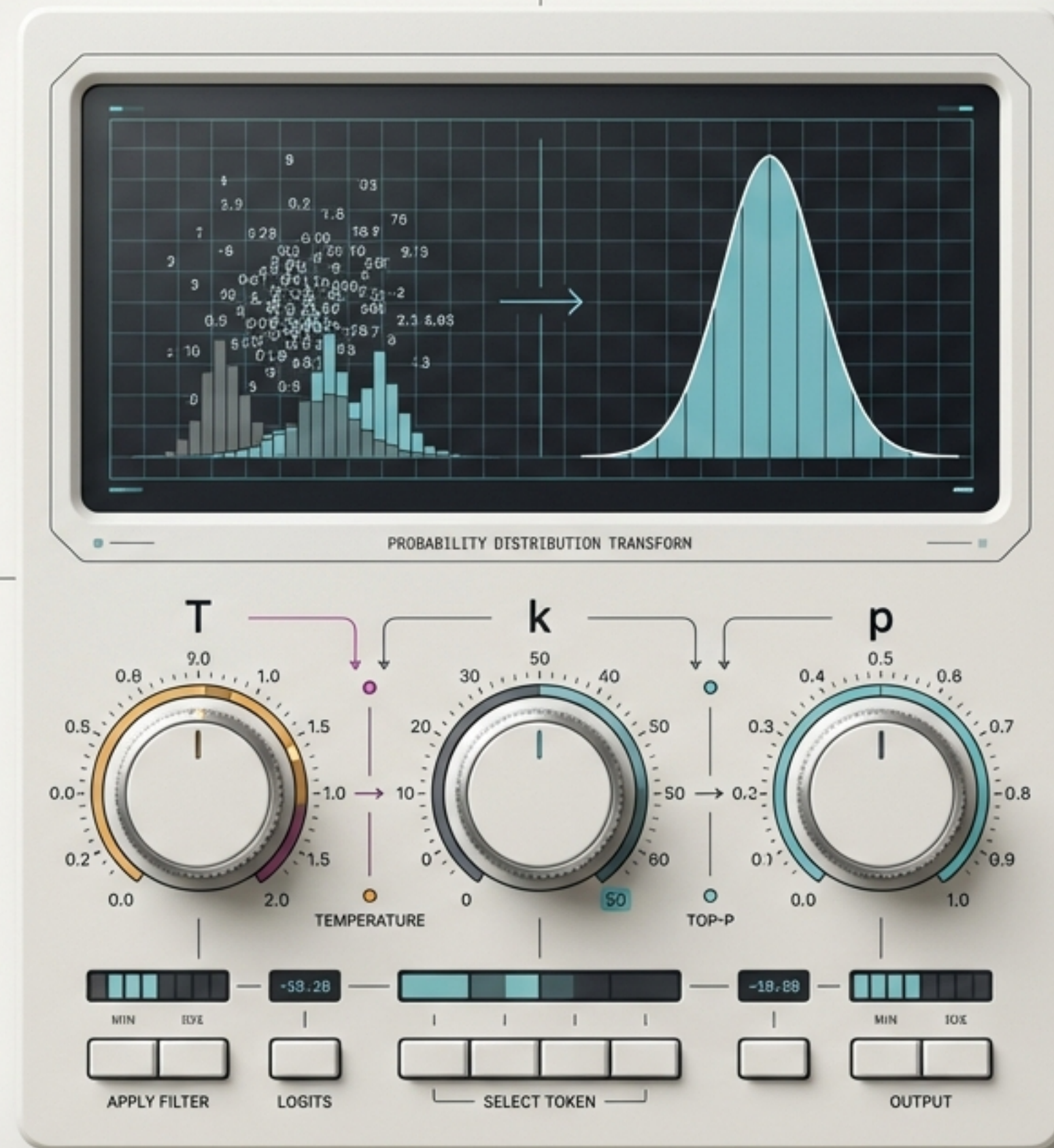


Секреты Семплинга: Как языковые модели выбирают слова

Архитектура принятия решений
под капотом LLM — от сырых
логитов до финального токена.





Сырой сигнал: Вектор логитов

На каждом шаге генерации нейросеть не выдаёт готовое слово. Она генерирует вектор логитов (z) — массив вещественных чисел, где каждому токenu из словаря присвоен свой «вес».

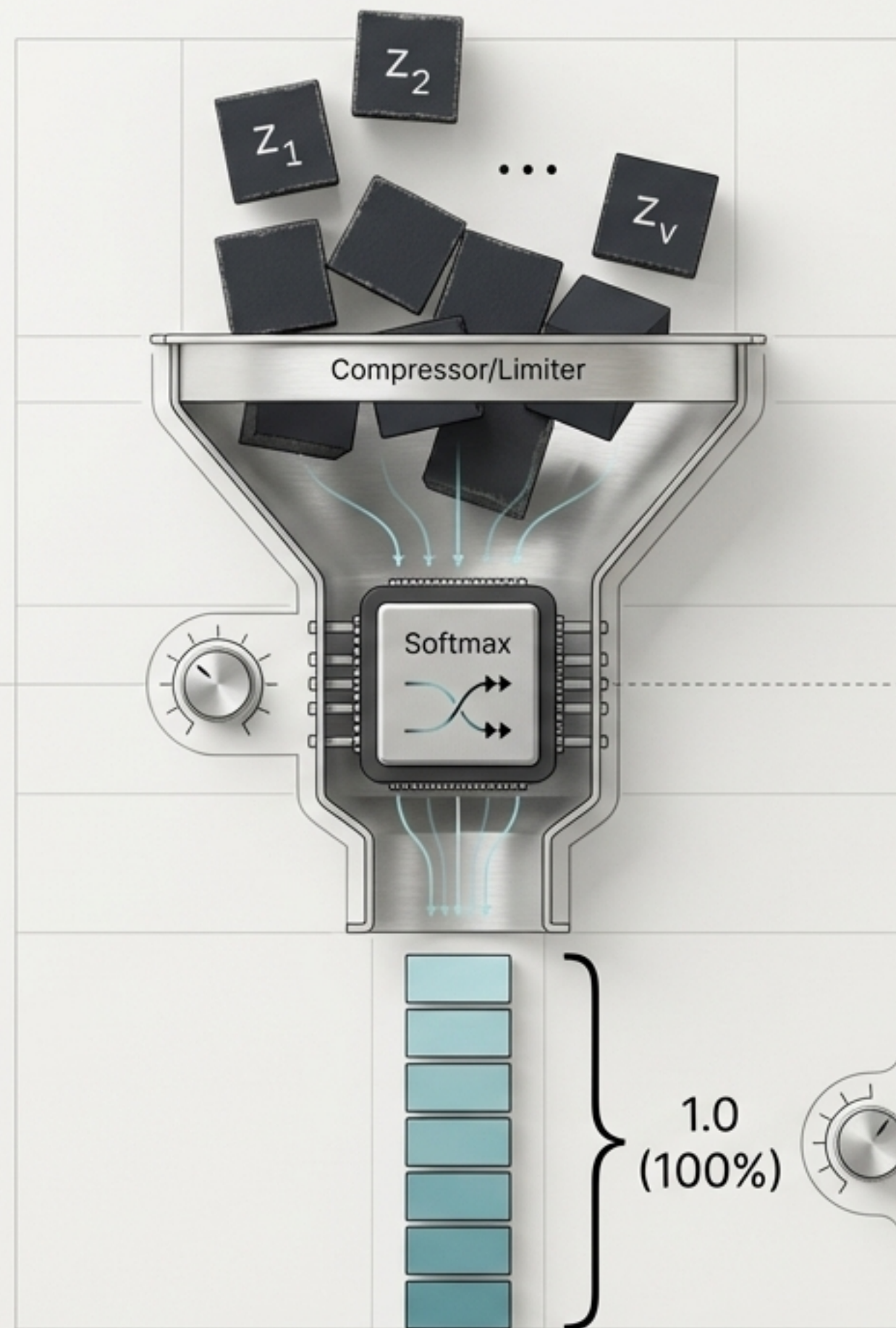
Логиты — это сырая метрика уверенности модели, требующая нормализации.

Характеристики сигнала

- Размер словаря (V): от 32 000 до 128 000 токенов.
- Диапазон: от $-\infty$ до $+\infty$
- Сумма значений: Не фиксирована.

Нормализатор: Функция Softmax

Чтобы превратить сырой сигнал в вероятностное распределение, применяется **Softmax**. Он делает все значения положительными (через экспоненту) и сжимает их так, чтобы их сумма равнялась ровно единице (100%).



$$P(w_i) = \frac{e^{z_i}}{\sum e^{z_j}}$$

1 — Гарантирует положительность

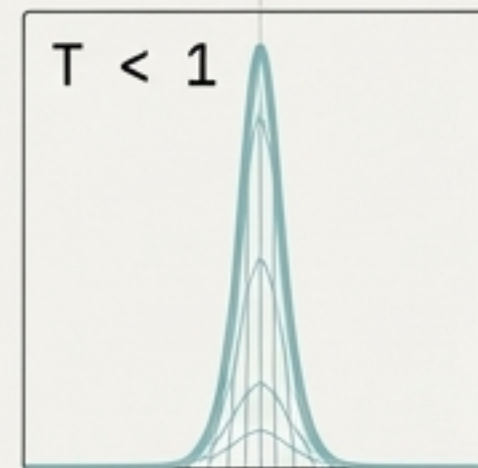
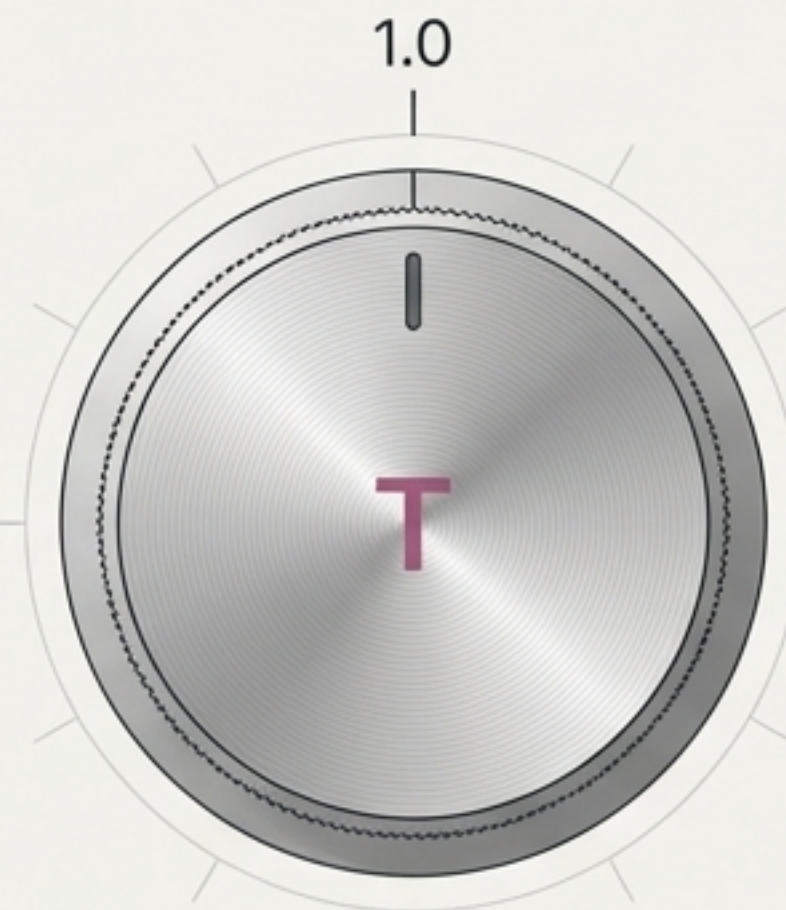
2 — Нормировочная константа (сумма всех экспонент)

Регулятор №1: Температура (T)

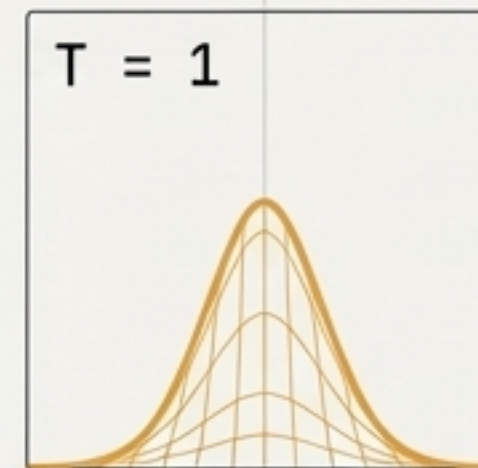
Температура управляет «уверенностью» модели. Это единственный параметр, который вмешивается непосредственно в математику Softmax, масштабируя логиты перед возведением в экспоненту.

$$P_T(w_i) = \frac{e^{z_i/T}}{\sum e^{z_j/T}}$$

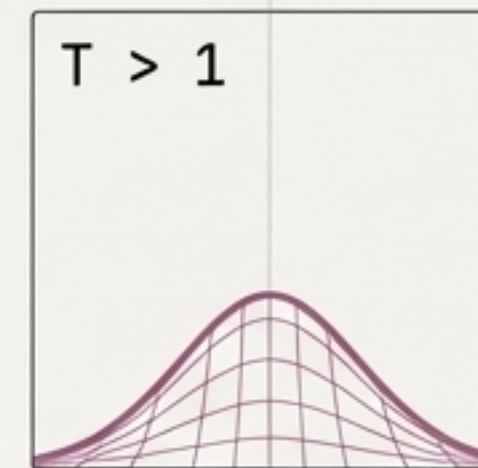
Деление на T выделено как главный рычаг искажения.



Острое
распределение.
Логиты растут.



Базовое
распределение.

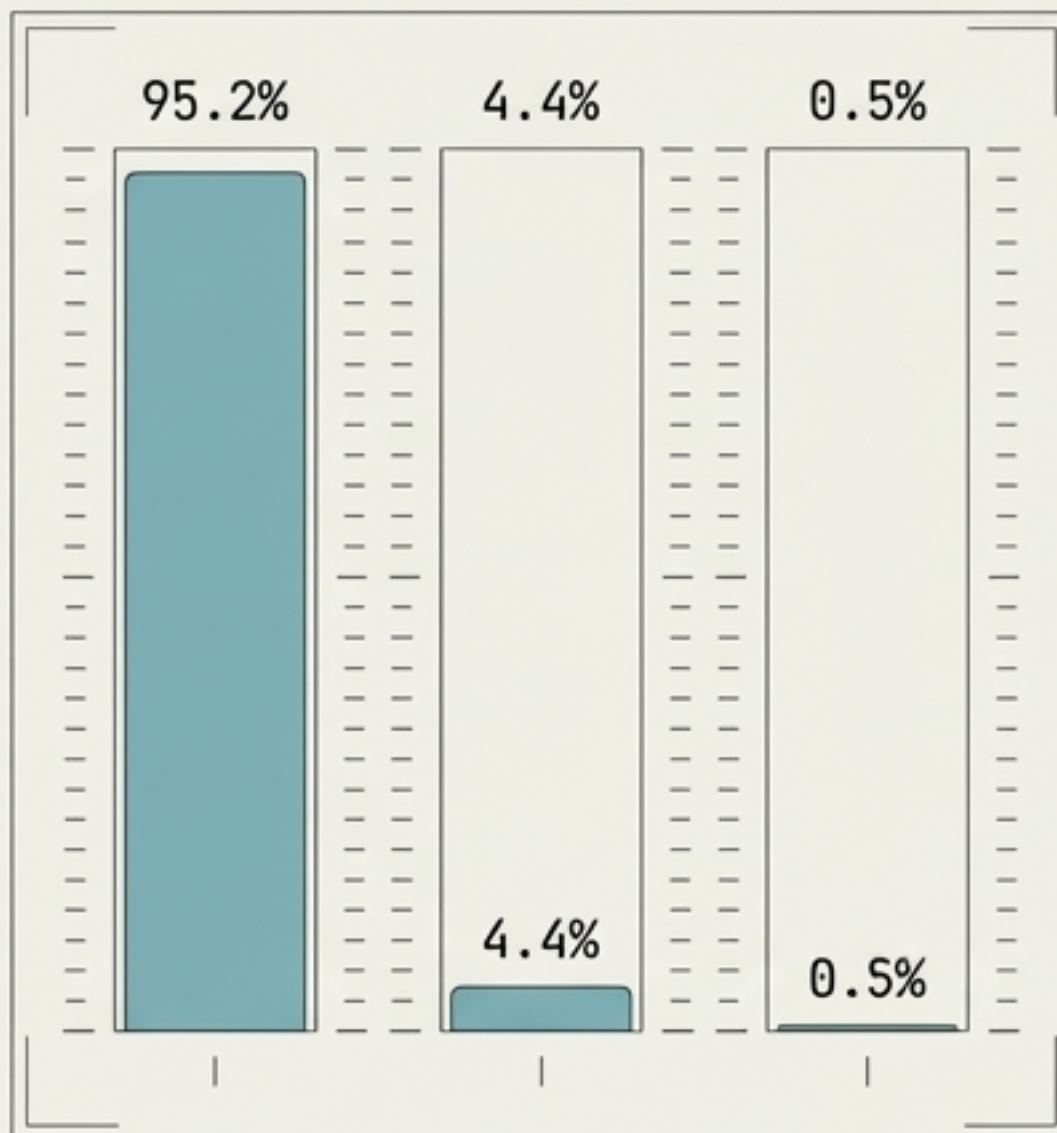


Плоское
распределение.
Шансы выравниваются.

Температура в действии: Искажение сигнала

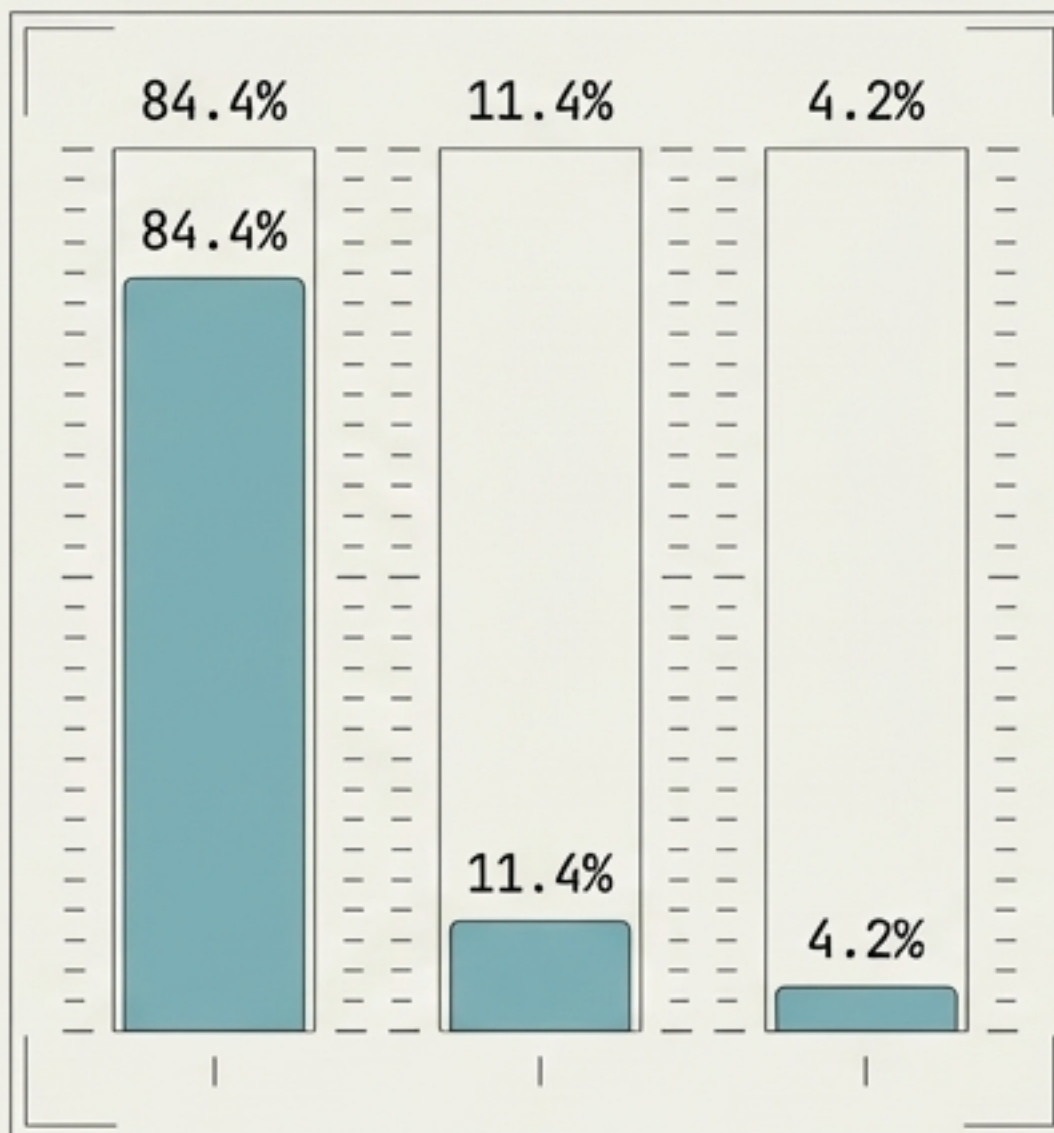
Трансформация одного вектора логитов $z = [3.0, 1.0, 0.0]$ при различных значениях T .

$T = 0.5$ (Высокая уверенность)



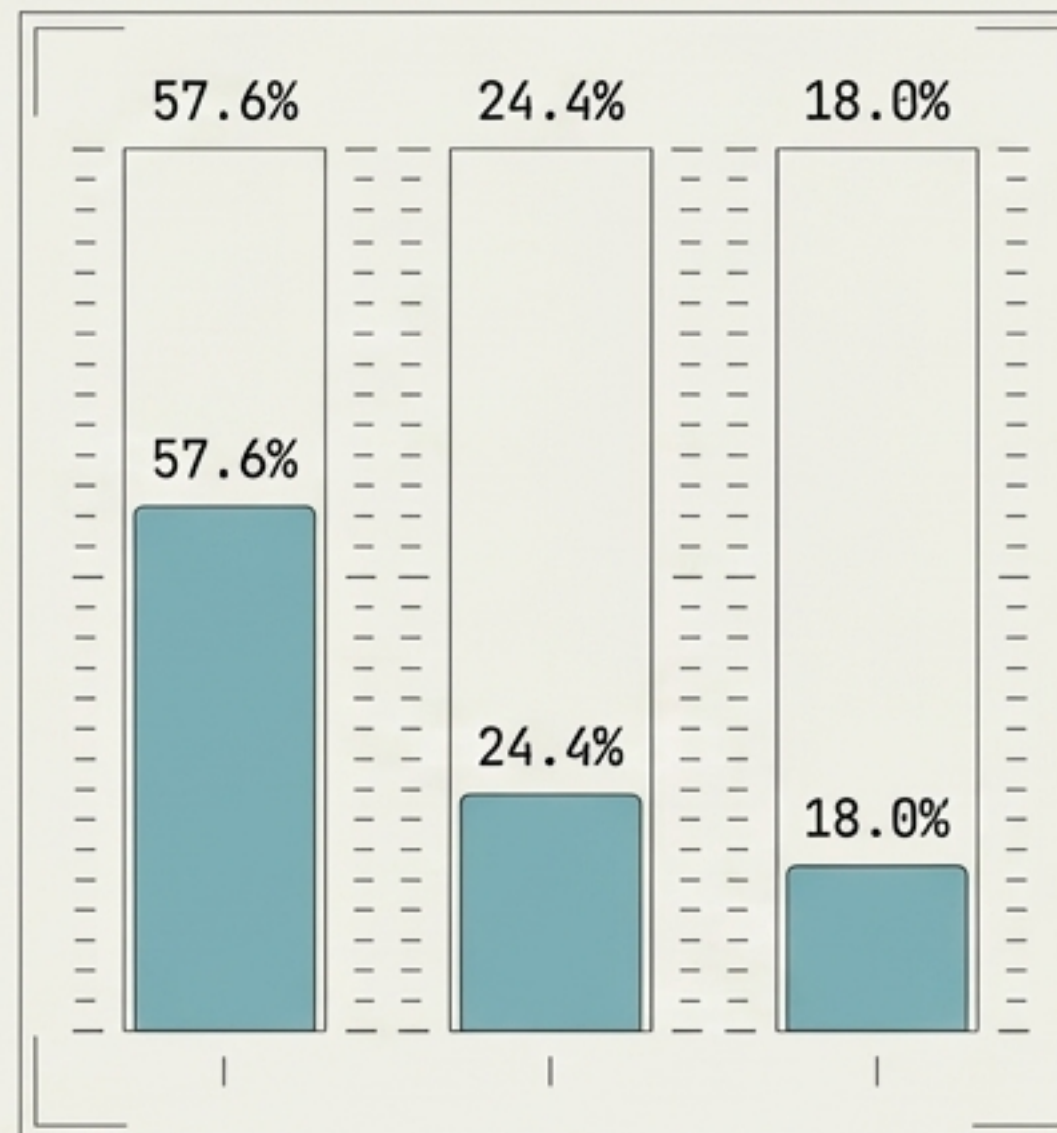
Токен №1 почти полностью подавляет остальные варианты.

$T = 1.0$ (Оригинал)



Стандартное распределение вероятностей, записанное при обучении.

$T = 2.0$ (Высокий хаос)



Шансы выравниваются. Модель готова выбрать маловероятный токен.

Проблема «Длинного хвоста»

Даже при низкой температуре в хвосте словаря остаются тысячи токенов с крохотными (например, 0.0001%), но ненулевыми вероятностями.



Регулятор №2: Top-k (Жесткое усечение)

Радикальное решение проблемы хвоста. Алгоритм оставляет только фиксированное число (k) лучших кандидатов, полностью обнуляя вероятности всех остальных токенов.

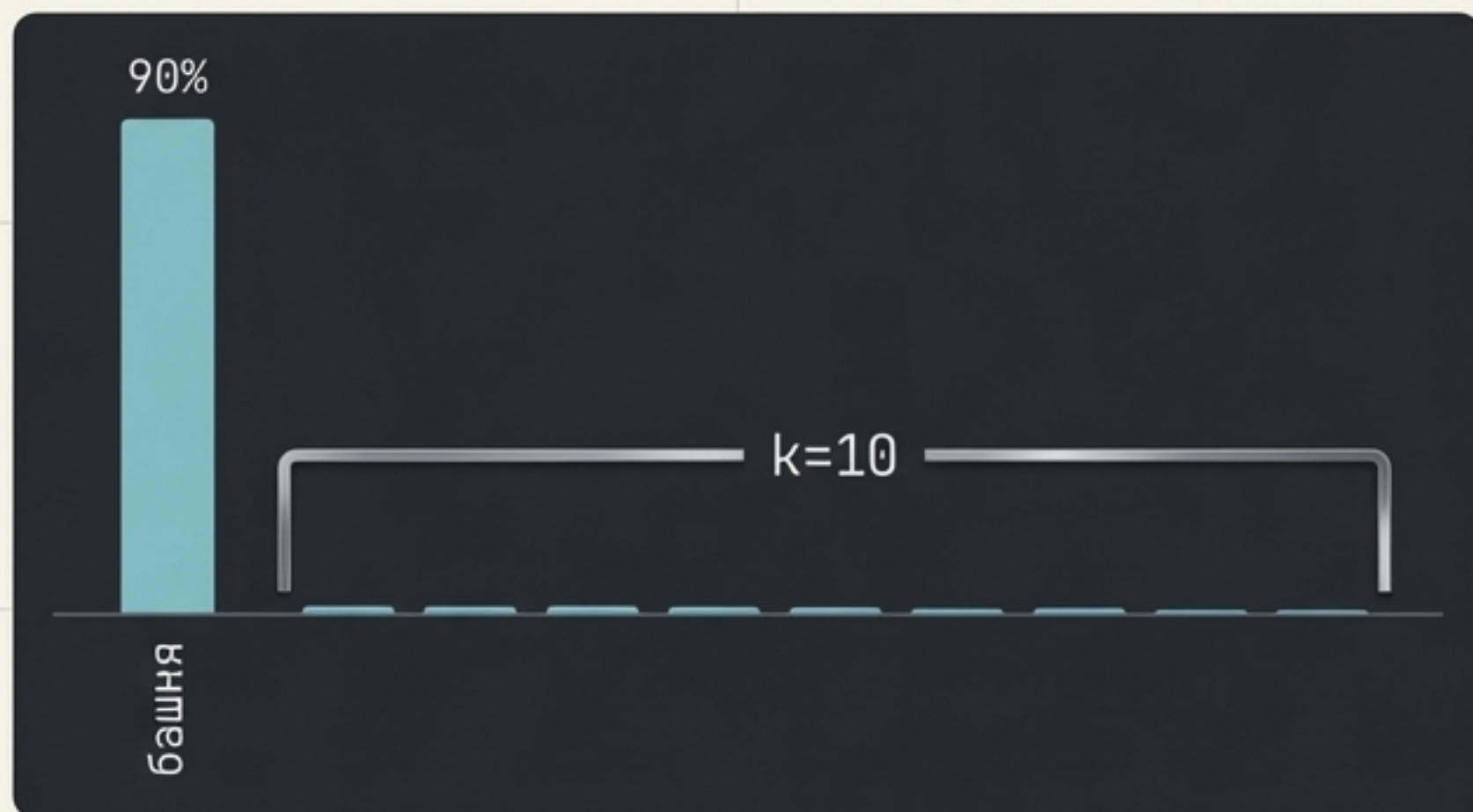


Иллюзия фиксированного числа: В чем слабость Top-k?

Жесткое ограничение k не умеет адаптироваться к изменяющемуся лингвистическому контексту.

Жесткий контекст: Очевидное продолжение

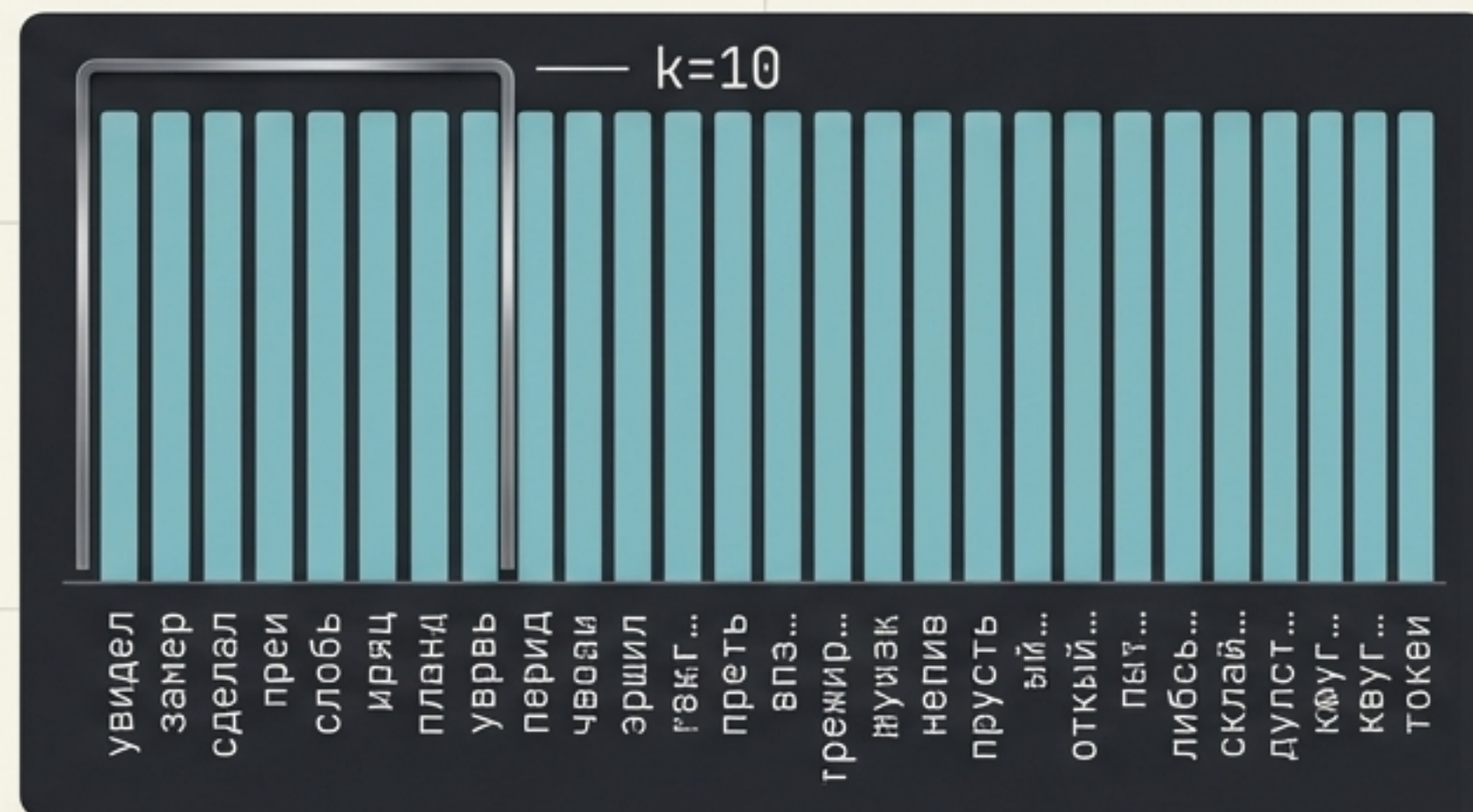
[Контекст]: Эйфелева...



Здесь $k=10$ — это слишком много. Мы искусственно впускаем 9 мусорных токенов в выборку.

Открытый контекст: Высокая неопределенность

[Контекст]: Он открыл дверь и...

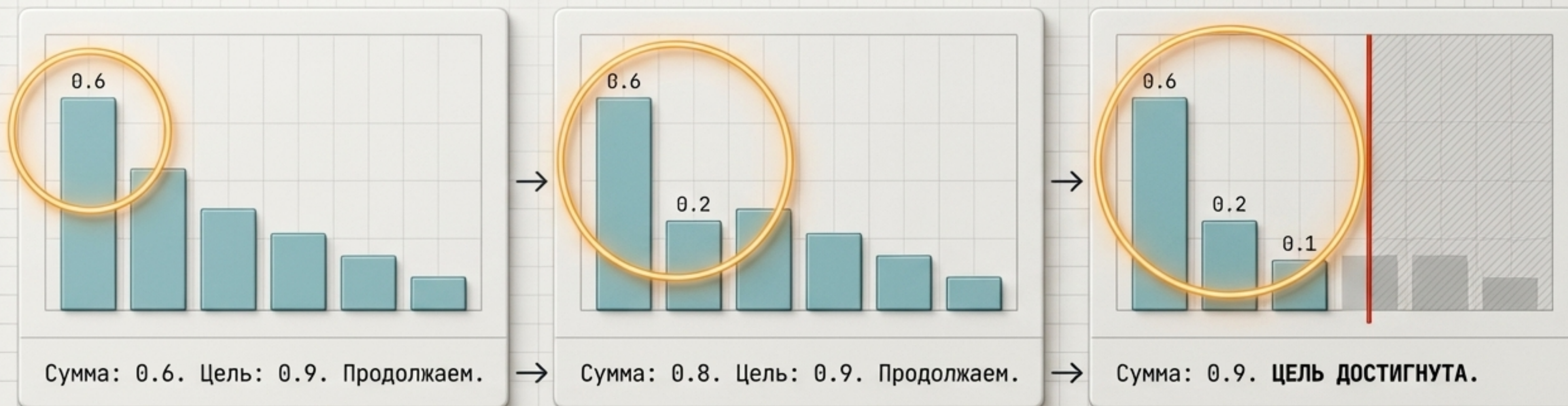


Здесь $k=10$ — слишком мало. Мы необоснованно убиваем креативность и легитимные варианты.

Регулятор №3: Top-p (Nucleus Sampling)

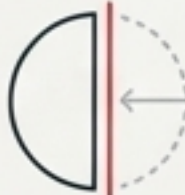
Вместо фиксации количества токенов, мы фиксируем суммарную вероятностную массу (p). Ядро динамически расширяется, захватывая токены, пока их общая сумма не достигнет порога (обычно 0.9).

$\sum P(w_i) \geq p$. При остром распределении ядру хватит 2–3 токенов. При плоском — расширится до сотен.

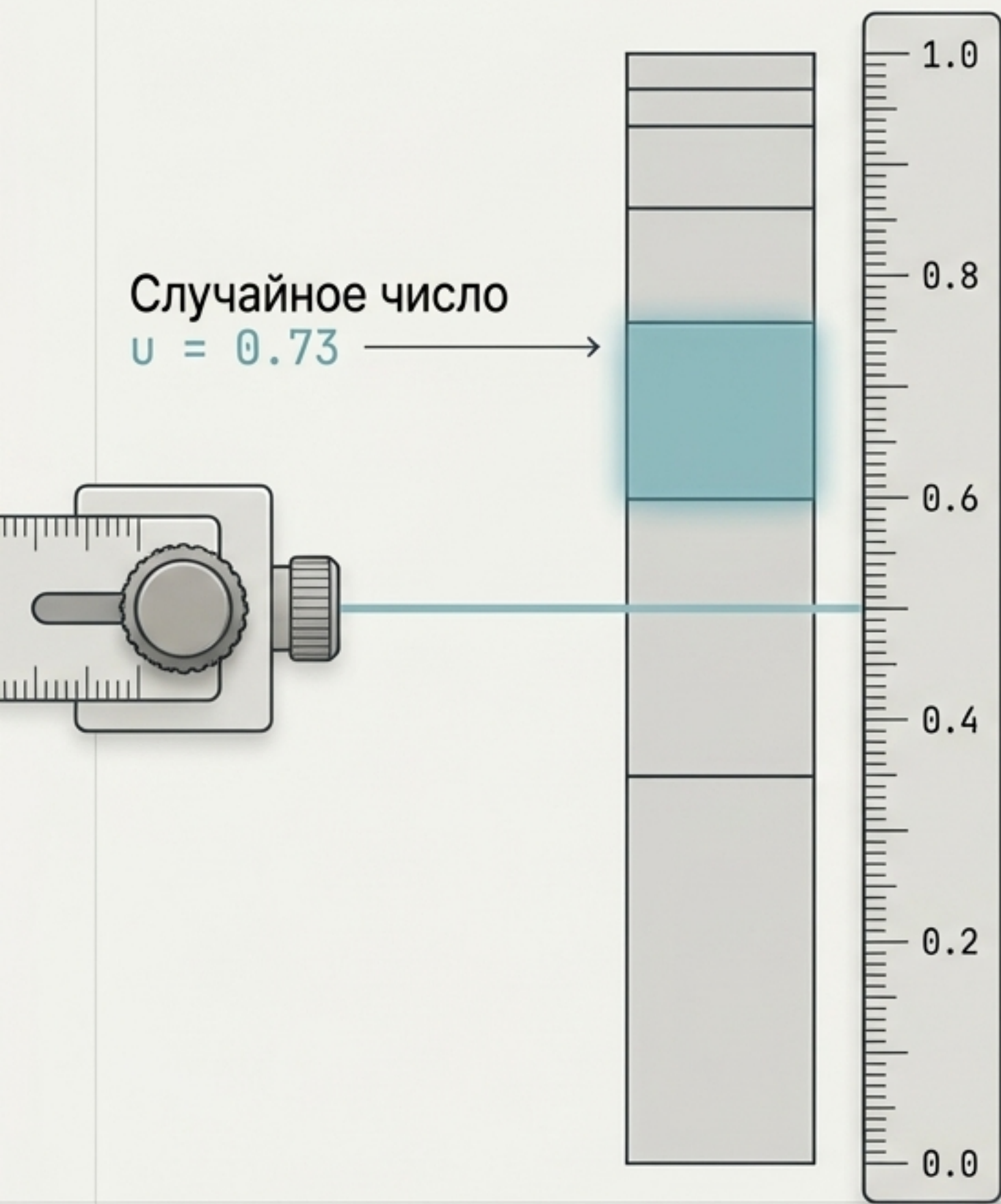


Сравнительная матрица: Методы усечения хвоста

Анализ двух подходов к фильтрации словаря перед финальным броском кубика.

Характеристика	Top-k  Alert Red	Top-p / Nucleus  Warm Amber
Механика работы	Ограничивает по индексу (жесткий счетчик) (k)	Ограничивает по накопленной вероятности (масса p)
Поведение при смене контекста	Размер словаря всегда строго равен k	Размер пула динамически пульсирует от 1 до N
Риск при плоском распределении	Искусственно отрезает валидные, логичные варианты	Расширяет ядро, сохраняя креативное разнообразие

Тор-р стал индустриальным стандартом благодаря своей контекстной адаптивности. Однако на практике они часто комбинируются для двойной защиты.



Механика выбора: Инверсия CDF

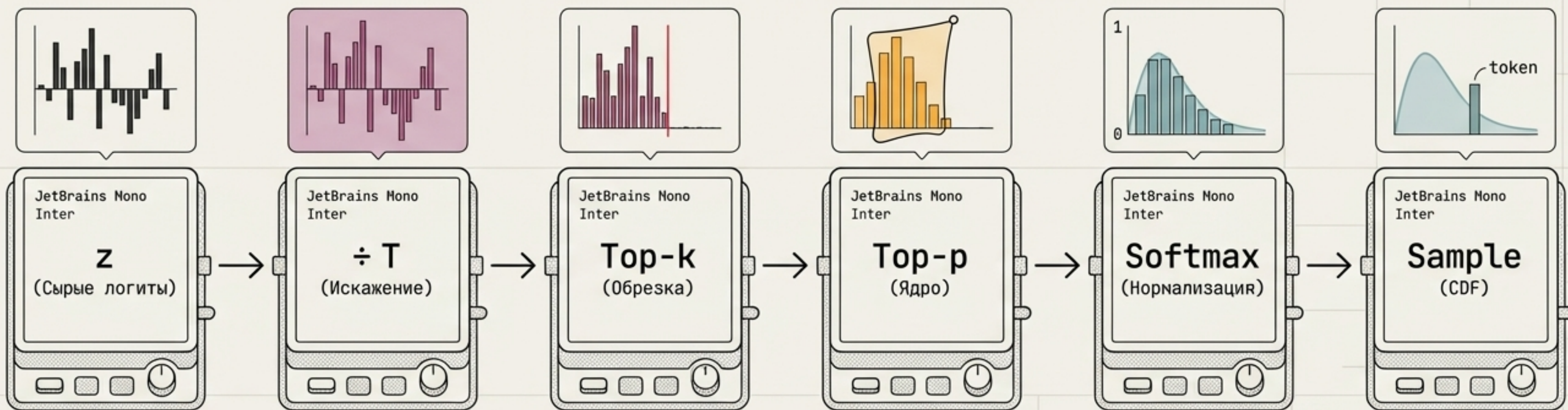
Все регуляторы лишь подготавливают распределение. Как машина делает финальный бросок кубика? Через кумулятивную функцию распределения (CDF).

1. Вероятности токенов выстраиваются в вертикальную стопку (сумма всегда = 1.0).
2. Генератор случайных чисел выдает равномерное значение u от 0 до 1 (луч сканера).
3. Выбирается первый токен в стопке, чья накопленная вершина пробивает этот луч.

Этот алгоритм математически корректно выбирает слово пропорционально его вероятности.

Синтез: Цепь прохождения сигнала

Параметры не исключают друг друга. Они образуют строгую последовательность post-processing операций над одним вектором логитов.



Памятка оператора: Готовые пресеты

Типичные рабочие настройки для управления компромиссом между детерминизмом и креативностью.

Точный ответ (Код, факты, математика)



T = 0.2



k = Выкл



p = Выкл

Почти жадный декодинг. Исключает галлюцинации.

Нейтральный текст (Статьи, письма)



T = 0.8



k = Выкл



p = 0.90

Идеальный баланс связности и естественного стиля.

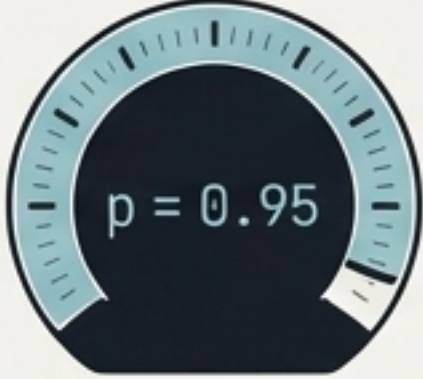
Творческое письмо (Стихи, сторителлинг)



T = 1.2



k = Выкл



p = 0.95

Высокая вариативность, логика удерживается широким ядром.

Максимальное разнообразие (Брейншторм)



T = 1.5



k = 50



p = Выкл

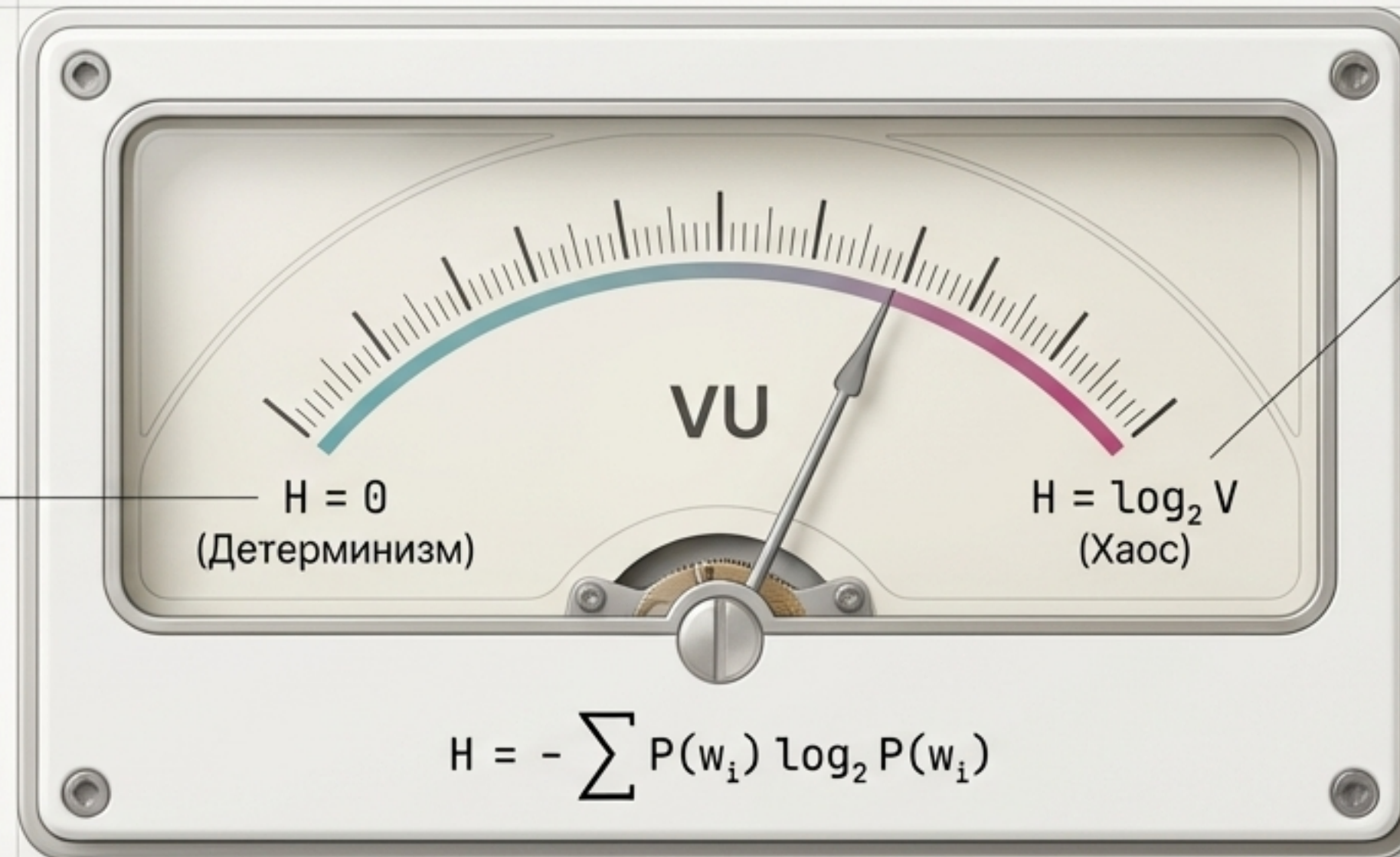
Экстремальный разогрев жестко сдерживается границей k.

Физика случайности: Энтропия Шеннона

Все рычаги семплинга управляют одной фундаментальной величиной — **энтропией** (неопределенностью) распределения.

Минимум ($H=0$):

При $T \rightarrow 0$
(greedy декодинг).
Никакой
неопределенности,
система всегда
выбирает одно слово.



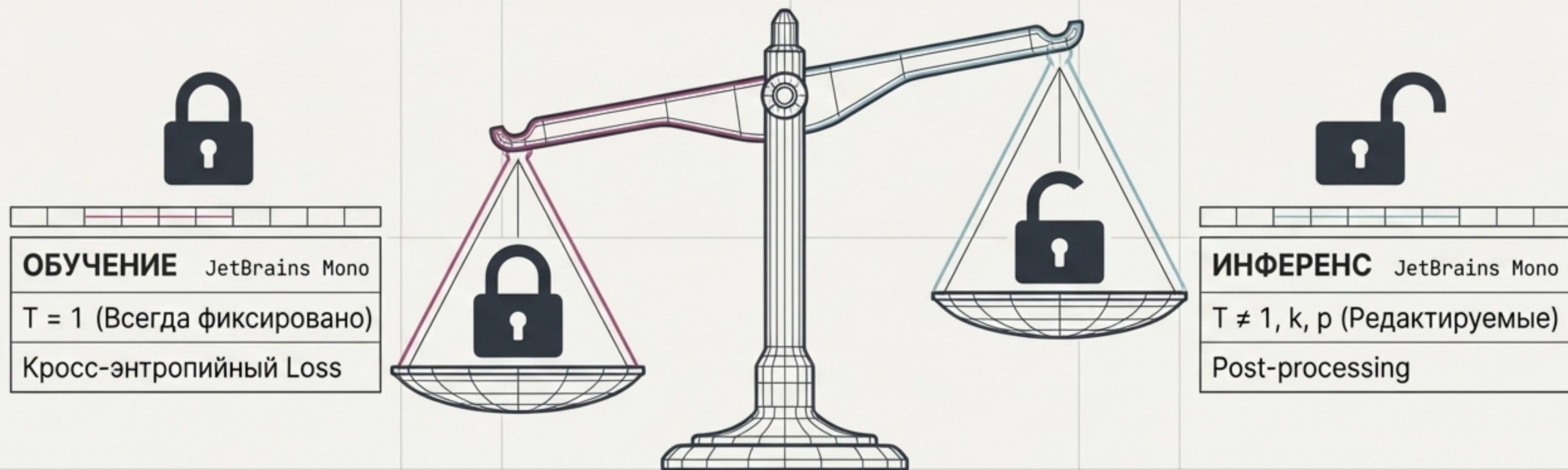
Максимум:

Абсолютно плоское,
равномерное
распределение.
Полный хаос.

Семплинг — это искусство точного позиционирования
стрелки энтропии на этой шкале под конкретную задачу.

Финальный парадокс: Инференс против Обучения

Меняя параметры семплинга, мы заставляем модель работать в режиме, который она никогда не видела при обучении.



Инсайт

Механизмы семплинга — это осознанное математическое «читерство». Мы модифицируем сырой сигнал поверх весов модели, искусственно повышая детерминированность или шумность ради решения человеческих задач.